

可持久化与扫描线

——成都七中 nzhtl1477

1.可持久化

可持久化的定义

- 我们对于一个数据结构，想维护其所有的历史版本
- 部分可持久化：允许访问历史版本，不能修改历史版本
- 完全可持久化：允许把目前版本替换为历史版本

可持久化线段树

- 如果每次都暴力复制整棵线段树，时间复杂度无法承受
- 考虑每次修改，线段树只有 $O(\log n)$ 个节点会被修改，所以我们可以对每个被修改的节点，将其复制一份进行修改



可持久化线段树

- 可以发现这样做，我们时间复杂度和空间复杂度都变成了 $O(m \log n)$ ，比起暴力复制改善了很多
- 很多情况下，如果题目没有强制在线，那么我们可以用分治来解决，而不用可持久化

经典问题

- 给一个序列，每次查询区间的k小值

值域线段树

- 定义值域线段树：我们在离散的值域上建立一棵线段树，可以考虑预先进行离散化来缩小值域，这时线段树每个节点表示值在一个范围内的数，在这个例子中我们只需要统计一个范围内有多少数。

Solution

- 我们先维护出每个前缀的值域线段树，这个可以用刚才讲的可持久化的方法
- 每次从前缀 $[1, t]$ 拓展到 $[1, t+1]$ 的时候，即在 t 版本的值域线段树上插入 $a[t+1]$ 这个值，并将这个版本记做第 $t+1$ 个版本
- 发现可以用之前讲的技巧进行优化，使得复杂度做到 $O(n \log n)$

Solution1

- 我们维护出了每个前缀的值域线段树，考虑怎么求出kth
- 发现可以二分答案，然后转化为区间中小于x的数个数
- 我们可以用r位置和l-1位置的值域线段树差分，即在两个历史版本的线段树上都查询小于x的数个数，来知道区间小于x的数个数
- 总时间复杂度 $O(n \log n + m \log^2 n)$

Solution2

- 在一棵值域线段树上，我们可以二分来求出kth
- 由于值域线段树结构是相同的，所以可以很方便地一起二分
- 每次传两个指针，表示当前走到的r位置树的节点a，和当前走到的l-1位置树的节点b
- 每次二分的时候，判断当前的k和 $a \rightarrow \text{left} \rightarrow \text{size} - b \rightarrow \text{left} \rightarrow \text{size}$ 大小即可，和普通的线段树上一起二分类似
- 总时间复杂度 $O((n+m)\log n)$
- 存在 $O((n+m)\log n / \log \log n)$ 的解法

Path Copy

- 上述的方法即path copy，字面意思就是每次复制被修改的整条路径来实现可持久化
- 在竞赛中，一般来说path copy的功能是足够的
- 优点：方便理解，出题人也只会这个
- 缺点：空间有时候显得很大

Fat Node

- 我们每个节点开一个vector，如果这个节点被访问到了，则在这个节点的vector中push_back一个二元组信息，表示这个节点在某个时刻被更改为了一个状态
- 每次我们访问到一个节点的信息时，则在其vector上二分（不一定要这么做，vector可以换成其他数据结构，这里是为了简单理解），找到距离查询的时刻最近的一次修改之后这个节点的信息变成了什么，然后返回结果

Fat Node

- 可以发现这样修改复杂度 $O(\log n)$, 如果使用vector, 则查询复杂度为 $O(\log^2 n)$, 这个复杂度是难以优化的, 目前没有方法优化到 $O(\log n)$ 的复杂度
- 优点: 修改高效, 空间常数小
- 缺点: 查询低效

Note

- 还有其他很多可持久化的技术，比如“Fat Copy”，“Node Splitting”等，是学术界之前的研究成果，比我们OI里面做到的可持久化强了很多
- 从这点可以看出学术界在技术上是吊打我们的

HDU - 4575 Tree

- 给出一颗树，每棵树上共有 n 个结点，每个结点对应一个值，有 m 组查询操作，查询在从 x 到 y 的这条路径上选出一个数与 z 进行异或的最大值。

Solution

- 考虑如何类比前面的区间kth做法

Solution

- 给定一个集合，如何查询一个数 x 与集合的最大xor和？
- 对集合建立一棵可持久化01Trie（和值域线段树一样）
- 从高位到低位贪心，如果当前 x 这一位是0，则尽可能走1，否则尽可能走0

Solution

- 树上怎么做?
- 考虑树上前缀和的方法, 从根到叶子建立可持久化01Trie
- 每次查询的时候可以在 $O(1)$ 棵Trie上一起二分实现, 和区间kth是类似的
- 复杂度 $O((n + m)w)$

Bzoj3524 [Poi2014]Couriers

- 给一个长度为 n 的序列 a 。 $1 \leq a[i] \leq n$ 。
 m 组询问， 每次询问一个区间 $[l,r]$ ， 是否存在一个数在 $[l,r]$ 中出现的次数大于 $(r-l+1)/2$ 。
- 如果存在， 输出这个数， 否则输出0。

Solution

- 对序列建立可持久化Trie，每次查询的时候可以类比区间kth：
- a对应了 $[1, l-1]$ 的Trie，b对应了 $[1, r]$ 的Trie
- 如果a的左子树大小-b的左子树大小 $\geq$ 区间长度一半，则答案可能在这里
- 如果a的右子树大小-b的右子树大小 $\geq$ 区间长度一半，则答案可能在这里
- 否则答案不存在
- $O((n+m)\log n)$

Bzoj2653: middle

- 一个长度为 n 的序列 a ，设其排过序之后为 b ，其中位数定义为 $b[n/2]$ ，其中 a, b 从0开始标号,除法取下整。
- 给你一个长度为 n 的序列 s 。
回答 Q 个这样的询问： s 的左端点在 $[a, b]$ 之间,右端点在 $[c, d]$ 之间的连续子序列中，最大的中位数。其中 $a < b < c < d$ 。
- 强制在线

Solution

- 这种形式的查询如何处理呢？
- 不难发现实际上就是我们必须选出 $[b,c]$ 这个区间
- 然后两边可以各增加一个 $[l,b)$ 和 $(c,r]$ ，其中 l 在 $[a,b[$ 中， r 在 $[c,d]$ 中



Solution

- 中位数的通用处理方法？
- 二分答案 x ，把小于 x 的设为-1，大于等于 x 的设为1
- 如果区间和 ≥ 0 ，则 $\geq x$ 的数出现次数 $\geq$ 区间长度的一半，则中位数 $\geq x$

Solution

- 于是我们可以二分答案 x
- 验证的时候，先查询 $[b,c]$ 中 $\geq x$ 的数个数
- 然后需要找到左右两边和最大的一段？
- 从小到大加入权值，把小于当前权值的设为-1，否则设为1
- 用线段树维护区间和与最大的前后缀和，这样就可以找到一个点向两边最大延伸出的一段了
- 可持久化这个线段树即可
- $O(n \log n + m \log^2 n)$

【UNR #1】 火车管理

- 有 n 个栈, m 次操作。
- 1.将 x 压入 $[l,r]$ 的栈中
- 2.将 l 的栈顶弹出
- 3.询问 $[l,r]$ 栈顶的和
- $n, m \leq 5 \times 10^5$

Solution

- 第一个和第三个操作都是基本的线段树操作，第二个操作我们可以把它看做返回该元素的上一个版本，不难想到用可持久化线段树去维护。
- 我们在可持久化线段树上维护两个信息，当前节点的版本和当前区间的权值，这个版本标记我们可以把它看做lazy标记，每次访问的时候下放即可。
- $O(m \log n)$

可持久化数组

- 字面意思，我们需要完全可持久化一个数组，这里强制在线
- 在OI中，我们使用可持久化线段树来实现可持久化数组，时间复杂度 $O(\log n)$
- 这个问题被证明了复杂度为 $\Theta(\log \log n)$ ，因为使用vEB树或y-fast trie而not practical，具体做法不做详细介绍

Luogu3402 可持久化并查集

- 1 a b 合并a,b所在集合
 - 2 k 回到第k次操作之后的状态(查询算作操作)
 - 3 a b 询问a,b是否属于同一集合, 是则输出1否则输出0
- 请注意本题采用强制在线,所给的a,b,k均经过加密,加密方法为 $x = x \text{ xor } \text{lastans}$,lastans的初始值为0

可持久化并查集

- 可以使用可持久化数组来实现可持久化并查集
- 使用按秩合并的并查集，每次操作时需要访问 $O(\log n)$ 个数组位置
- 数组使用可持久化线段树来实现即可
- 时间复杂度 $O(\log^2 n)$
- 存在 $O(\log n)$ 的实现，但是在OI中没有价值

可持久化平衡树

- 我们的平衡树也是可以可持久化的
- 很多人关于可持久化平衡树都在瞎扯
- 1.带旋转的平衡树不能可持久化——错的
- 2.维护了父亲的平衡树不能可持久化——错的
- 3.可持久化treap功能是足够的——错的，至少我们不知道怎么做

可持久化平衡树

- 如果是插入删除版本的，则只需要在修改的路径上复制一遍即可，旋转的时候也需要复制一遍
- 为了方便，请自顶向下维护，并且不维护父亲节点
- 虽然维护父亲节点也可以无复杂度代价的可持久化，但是很复杂，而且常数很大，所以还是别维护了

可持久化平衡树

- 如果是分裂合并版本的，则分裂合并的时候新建节点即可，写起来和不可持久化的平衡树差不多，很方便
- 具体如何实现这里就不详细介绍了

可持久化平衡树能做什么

- 可以发现可持久化平衡树维护权值的话一般是不如可持久化trie的，因为后者好写，而且常数更小，而且复杂度也是 $O(w) = \Theta(\log n)$ 的
- 感觉最大的一个应用是做区间复制，比如把一个区间复制到另一个区间上这样的操作

可持久化平衡树能做什么

- 这样的操作有很大的局限性，因为我们平衡树的复杂度是 $O(\log n)$ ，而一次这样的操作就会使得 n 扩大一倍，所以到最后 n 会变成指数级大小，导致平衡树复杂度退化为 $O(m)$ ，这里还不考虑高精度造成的影响
- 虽然合并两个任意大小的平衡树可以做到 $O(1)$ ，但一旦涉及到修改，复杂度还是会炸掉
- 所以一般会限定序列长度

可持久化treap的缺点

- 需要注意的一点是，由于treap随机种子的问题，导致其在可持久化的时候会出一些问题
- 比如对一个节点进行自我复制的时候，会导致复制出来的节点和原节点的随机种子相同，这样复杂度会出错
- 如果按照size随机进行旋转，在一些特定的问题中也会出错，比如自我复制k次时复杂度会多log

HDU 6087 Rikka with Sequence

As we know, Rikka is poor at math. Yuta is worrying about this situation, so he gives Rikka some math tasks to practice. There is one of them:

Yuta has an array A with n numbers and he keeps a copy of the initial value of array A as A' ($A'_i = A_i$). Then he makes m operations on it.

There are three types of operations:

1 $l\ r$: Yuta wants Rikka to sum up A_i for all i in $[l, r]$.

2 $l\ r\ k$: Yuta runs the C++ code “for (int i=l;i<=r;i++) A[i]=A[i-k]” on sequence A . (The pascal version of the code snippet is “for i:=l to r do A[i]:=A[i-k]”).

3 $l\ r$: Yuta changes A_i back to A'_i , for all $i \in [l, r]$.

It is too difficult for Rikka. Can you help her?

Solution

- 区间复制k次如何解决？
- 可以倍增地复制，只要合并两个大小为n和m的节点复杂度是 $O(\log(n/m))$ 就可以保证复杂度
- 还有一种做法是类似之前讲的平衡树维护连续段
- 我们可以将区间复制也看做是一次类区间染色操作
- 每个平衡树节点额外维护出其是否是一次区间复制产生的，并且是由哪个节点复制了多少次
- 分裂的时候特判这种节点即可

Solution

- 这样直接做空间是 $O(n \log n)$ 的，还带大常数
- 可以使用smart pointer来减少内存，这里讲另一种方法
- 可以每 $\Theta(n/\log n)$ 次重构一次整个序列，并且回收内存
- 发现这样空间是： $\Theta(n/\log n) * O(\log n) = O(n)$
- 时间是 $m / \Theta(n/\log n) * n = \Theta(m \log n)$
- 时间复杂度 $O(m \log n)$ ，空间复杂度 $O(n + m)$

可持久化可合并堆

- 1.插入
- 2.删除最小值
- 3.查询最小值
- 4.合并两个堆
- 目前最优复杂度是除了2操作pop复杂度为 $O(\log n)$ 以外，其他操作复杂度均为 $O(1)$
- 可以完全可持久化，并且可以合并历史版本

可持久化可合并堆

Brodal Queue

- 这是什么？
- http://en.wikipedia.org/wiki/Brodal_queue
- 插入、查询最小值、合并、减小某个元素的值都是最坏复杂度 $O(1)$ 的，只有删除操作是 $O(\log n)$ 的。
- 并且它能够可持久化。
- 网上流传的一个关于可持久化可并堆的pdf
- 这个说法也是错的，可持久化的Brodal Queue无法减小某个元素的值，只能做上述的四个操作，而且Brodal论文中给出的是一个新的数据结构，叫做斜二项堆
- 如果对Brodal queue感兴趣可以看<https://www.luogu.com.cn/problem/P6019>

Luogu3293 [SCOI2016]美味

- 一家餐厅有 n 道菜，编号 $1 \dots n$ ，大家对第 i 道菜的评价值为 $a_i (1 \leq i \leq n)$ 。有 m 位顾客，第 i 位顾客的期望值为 b_i ，而他的偏好值为 x_i 。因此，第 i 位顾客认为第 j 道菜的美味度为 $b_i \text{ XOR } (a_j + x_i)$ ，XOR 表示异或运算。
- 第 i 位顾客希望从这些菜中挑出他认为最美味的菜，即美味值最大的菜，但由于价格等因素，他只能从第 l_i 道到第 r_i 道中选择。请你帮助他们找出最美味的菜。

Solution

- 按照数位一位一位贪心，因为整体加了一个 x ，所以我们考虑对于所有的 $(a_i + x)$ 与 b 的按位异或。

Solution

- 假设我们已经处理到 b 二进制下的第 i 位，假设是1（0同理），
- 那么我们只需要查找是否存在 $a_j + x$ 使得其二进制第 i 位数字是0，我们已经处理了前 $i-1$ 位了，设当前结果是 ans ，那么我们需要查找的数的大小就是区间 $[ans - x, ans + (1 \ll i) - 1 - x]$
- 那么现在我们就是要在 $a[1] \dots a[n]$ 中找出是否存在于 $[ans - x, ans + (1 \ll i) - 1 - x]$ 的数字，可以使用区间小于 x 的数个数来做
- 总时间复杂度 $O(n \log n + m \log n \log v)$ ，可以除 $\text{polyloglog} n$

某经典问题

- 给一个DAG，每个边有边权，每次查询从一个点出发到任意点，字典序kth的路径，是到哪个点
- 每个点的每种边权的出边只有一条
- 强制在线

Solution

- 每个节点用一个可持久化平衡树维护答案
每次在DAG上按拓扑顺序合并一个点到的所有边的平衡树，每个平衡树是按原顺序合并过来的
- 然后这个合并的时候取前 $\max k$ 个节点就行了，不然是指数级的

Bzoj3551 Peaks加强版

- 有 n 个座山，其高度为 h_i 。有 m 条带权双向边连接某些山。多次询问，每次询问从 v 出发 只经过边权 $\leq x$ 的边 所能到达的山中，第 K 高的是多少。
- 强制在线

Solution

- 我们把平衡树启发式合并或者trie合并的过程可持久化一下
- 注意到这个可持久化是部分可持久化，只需要访问历史版本，不需要在历史版本上面修改，所以复杂度是不变的
- $O((n+m)\log n)$

某经典问题

- 查询两个树链sort之后有哪些位置不相同，保证输出量 $1e6$

Solution

- 使用可持久化值域线段树维护链hash值，这样每次相当于找出第一个出现次数不同的值
- 每次查询的时候在可持久化Trie上面二分这个，可以 $O(\log n)$ 的时间内找出一个不同的位置
- 然后暴力一个一个找下去就可以了
- 总复杂度 $O(n \log n + t \log n)$

Luogu P3302 [SDOI2013]森林

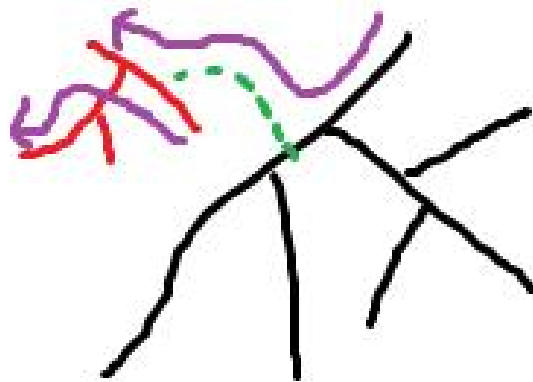
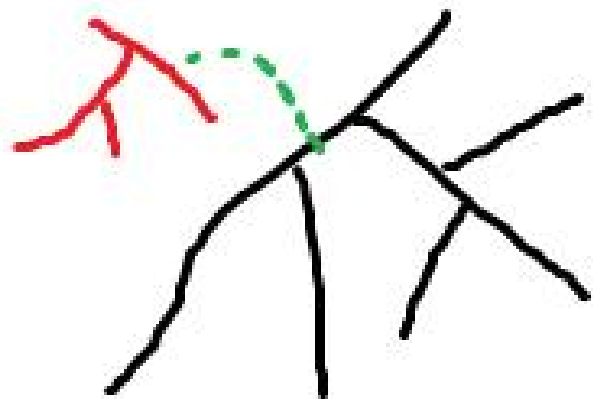
- 给一个森林，每个点有点权
- 1.link x y ，如果 x 和 y 已经连通无视这个操作
- 2.find x y ，查询 x 到 y 的链kth
- 强制在线

Solution

- 由于这个问题只加边，所以可以考虑一下均摊的方法
- 只加边，维护连通块信息该怎么做呢？
- 启发式合并
- 于是这一题我们可以考虑用启发式合并来处理

Solution

- Link x y 的时候, 从size小的那个连通块那里开始DFS, 维护树上的可持久化Trie, 相当于把小的连通块的父亲设为大的连通块
- 同时需要维护个倍增表或者LCT啥的
- 查询的时候就是普通的链kth



- $O(n \log^2 n + m \log n)$

2.扫描线

扫描线



我感觉扫描线是设计一个连续变化，且只发生有限个事件的过程。只需按顺序处理这些事件，而不用尝试模拟“连续变化”。

扫描线

- 在数据结构里面的应用基本上都是扫一个端点，维护另一个端点的答案
- 实际上扫描线是联系数据结构与计算几何的桥梁
- 很多扫描线的维护的题的计算几何意义更加浓厚

扫描线

- 可以将大量扫描线类题目抽象为：
- 在 k 维空间上进行若干次操作
- 操作结束后
- 在 k 维空间上进行若干次查询

Luogu1972 [SDOI2009]HH的项链

- 查询区间中颜色数

Solution

- 记下每个点之前最近的一个和其颜色相同数的位置
- 于是转换为：有一个二维平面上有 n 个点，每次查询矩形和
- 扫一个端点，每次相当于一段区间加，维护另一个端点的答案即可

Solution

- 每个数通过记录上一次出现的位置转换为一个二维点
- 每次查询区间 $[l, r]$ 中不同数个数等价于查询矩形 $[l, r][0, l-1]$ 点数
- 矩形数点是可以差分的，相当于查询前缀矩形（二维前缀和） $[0, r][0, l-1] - [0, l-1][0, l-1]$ 这两个矩形点数的差

Solution

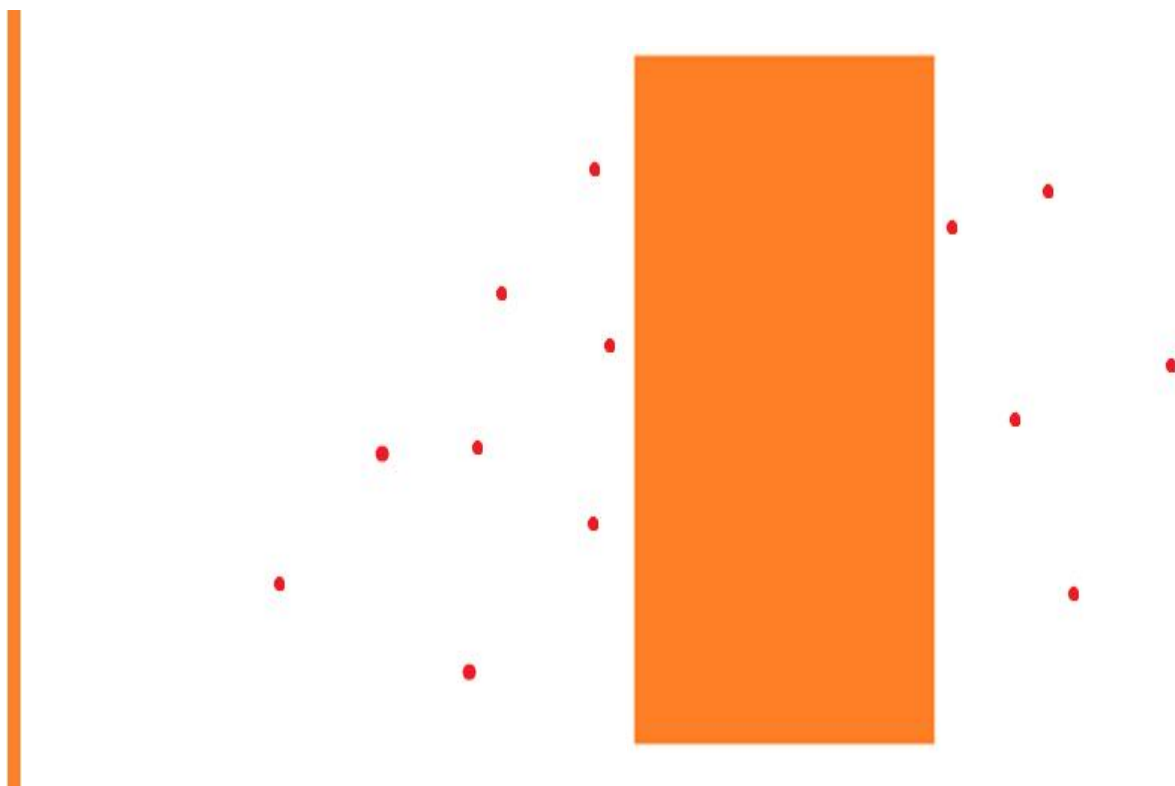
- 然后跑前缀，维护一个树状数组
- 比如现在在 i 点， i 点的第二维值是 j
- 就把 j 插入树状数组中
- 预先把查询挂链表挂在一个前缀的位置
- 然后等跑到这个位置之后查询即可

Solution2

- 还是记录下每个数上次出现的位置
- 预先把所有询问挂在右端点所在的位置，等扫描线跑到这个右端点的时候一起处理
- 然后扫描线扫右端点，用数据结构维护所有左端点的答案

Solution

- 扫描线其实是这样在跑:



Unity

- 我们通常认为的扫描线：扫一个端点，维护另一个端点的答案
- 其实等价于：把扫的这一维当作时间维，然后查询一个区间的历史信息合并
- 这个题经过扫描线后，由一个二维数点问题转换为了一个一维的区间历史和问题
- 如果历史信息可以高效维护，那么就实现了降维

Luogu4113 [HEOI2012]采花

- 查询区间中出现次数 ≥ 2 的颜色数

Solution

- 这题和区间颜色数真的有什么区别吗。。。。

Luogu 3268 [JLOI2016]圆的异或并 &BZOJ2758 [SCOI2012]Blinker的噩梦

- 平面上有 n 个多边形（凸包和圆）
- 任意两个多边形 AB 只有两种关系：（1） A 包含 B 或者 B 包含 A ；（2） AB 的公共面积为0。
- 每个多边形有一个值 x 。 m 个查询。
- 分两种：（1）修改某个多边形的值；
- （2）从一点 s 走到另一点 t 。每次走出一个多边形或者进入一个多边形时，都要xor上该多边形的值。输出走到 t 时的值。（由xor的性质和本题定义可得这个值跟走的路径无关）
- ----简洁版题面摘自某博客
- <https://blog.csdn.net/u014609452/article/details/50926290>

Solution

- 只要是那种给你一些图形，这些图形要么相互包含，要么相离，那就是扫描线把这些图形构成的树建出来，然后再建出来的这个树上面搞的题了
- 没有任何思维难度，但是会很码的无聊东西
- 这两个题一样处理就行

[Ynoi2015]た と え 未 来 が 見 え な く て も

- 查询一个区间里面长为1 -> 10的极长值域连续段的个数
- 这个查询是和每个数在区间中位置无关的
- 然后如果(l,r)是一个极长值域连续段，那不存在(l-1,r), (l,r+1)这两个值域连续段
- 比如这个是区间的所有值映射到值域数组上：
- 0 0 0 1 0 0 1 1 1 0 1 0 0 1 1 1 0 1 1 1 1 1 0 1 0
- (4,4)(7,9)(11,11)(14,16)(18,22)(24,24)就是所有的极长值域连续段
- 每次输出区间长为1,2...k的极长值域连续段段数
- $n,m \leq 2e6$ $k=10$ 3s

Solution1

- 暴力
- 总复杂度 $O(nm)$

Solution2

- 莫队+bitset
- 莫队跑出区间的值域bitset, 然后用四毛子来数区间每个值域连续段即可
- 总复杂度 $O(nm/w)$

Solution3

- 莫队+数据结构
- 用数据结构维护每个值域连续段, 每次莫队转移的时候可以在数据结构上二分实现
- 总复杂度 $O(n\sqrt{m}\log n)$

Solution4

- 不删除莫队
 - 维护每个极长值域连续段向左，右的最远拓展的位置
 - 每次如果只插入的话，可以 $O(1)$ 维护
 - 然后套用不删除莫队即可
-
- 总复杂度 $O(n\sqrt{m})$

Solution5

- 莫队+位运算优化
- 每次查询相当于是查询一个点往前，后有多少个连续的1
- 然后距离超过10就无意义了
- 这个可以用位运算优化
- 单次复杂度就是 $O(k/w) = O(1)$ 的了
- 总复杂度 $O(n\sqrt{m})$

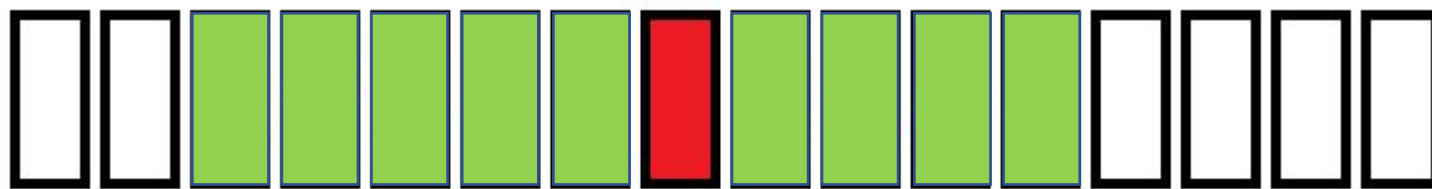
Solution6

- 考虑扫描线
- 扫右端点，开k个树状数组表示每个左端点的每个长度的极长值域连续段的个数
- 维护每个值最近的出现位置（在序列上的）
- 扫到一个位置的时候，暴力扫左，右k个



Solution6

- 相当于是把值在左右各k个的位置提出来，然后按照上次出现次数从大到小排序，然后一个一个插回来



- 这里就是左端点 ≤ 5 的时候出现了一个长为2的极长值域连续段
- 然后左端点 ≤ 3 的时候出现了一个长为3的极长值域连续段
- 然后左端点 ≤ 2 的时候出现了一个长为7的极长值域连续段
- 然后左端点 ≤ 1 的时候出现了一个长为10的极长值域连续段

Solution6

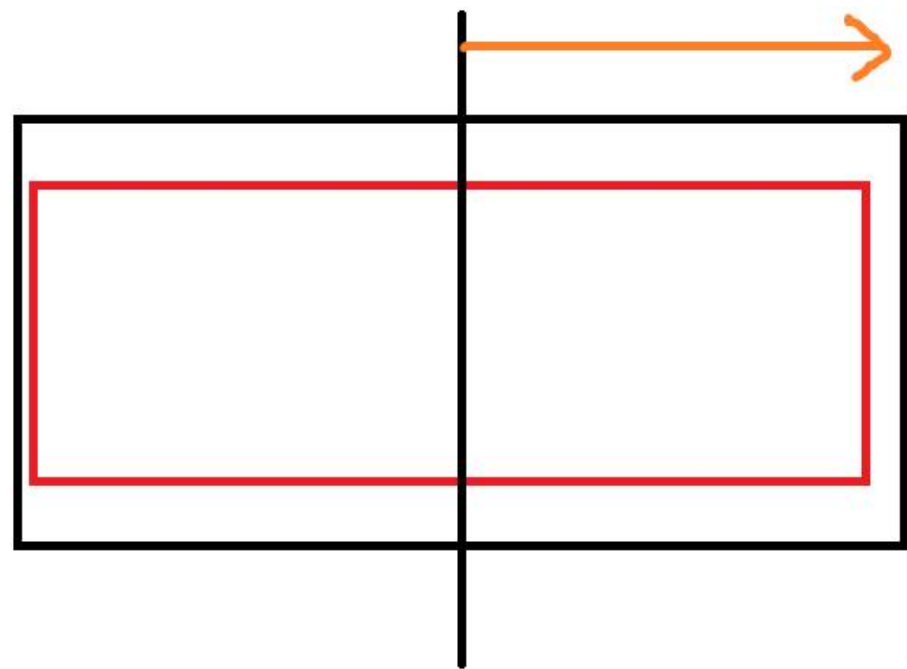
- 于是相当于是：
- 在这个新插入的点周围的长为2的极长值域连续段出现的左端点位置是[4,5]
- 长为3的极长值域连续段出现的左端点位置是[3,3]
- 长为2的极长值域连续段出现的左端点位置是[2,2]
- 长为1的极长值域连续段出现的左端点位置是[1,1]
- 然后把原来已经加了的贡献去掉，用这个新贡献替换就可以了
- 每次最多产生20个新的贡献
- 总复杂度 $O(k(n+m)\log n)$

[Ynoi2009]rprm_q

- 给定 $n \times n$ 的矩形，有 m 次矩形加，在所有修改结束后有 q 次矩形max查询

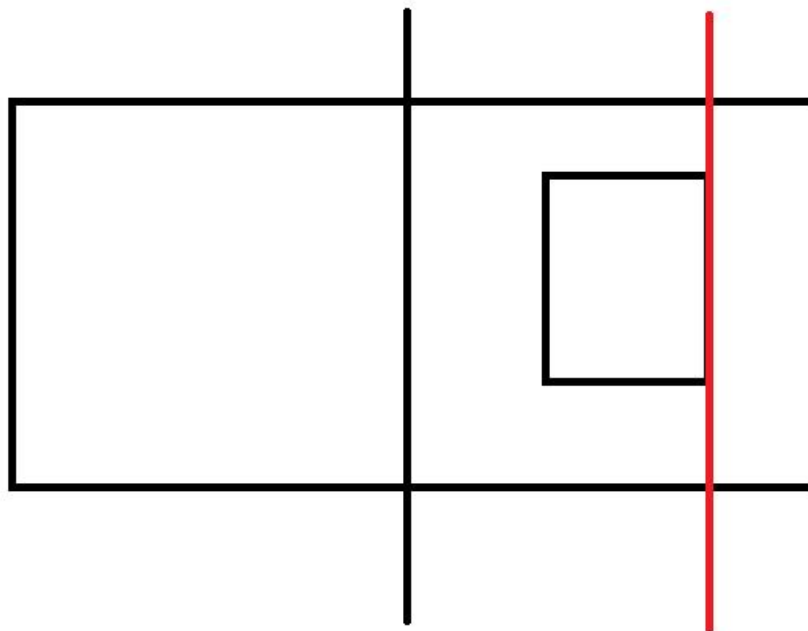
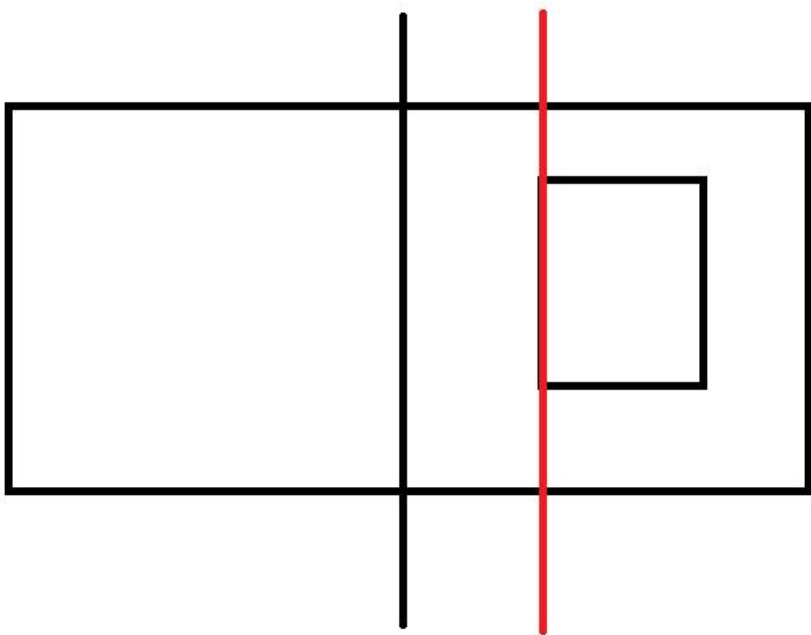
Solution

- 考虑对平面进行分治，选一维 x 维进行分治
- 每次分治的时候，处理跨过分治中线的询问，这样询问就变成一个 x 维前缀， y 区间合并的形式
- 分治的意义是拆出两个前缀，
- 因为这个信息不具有可减性



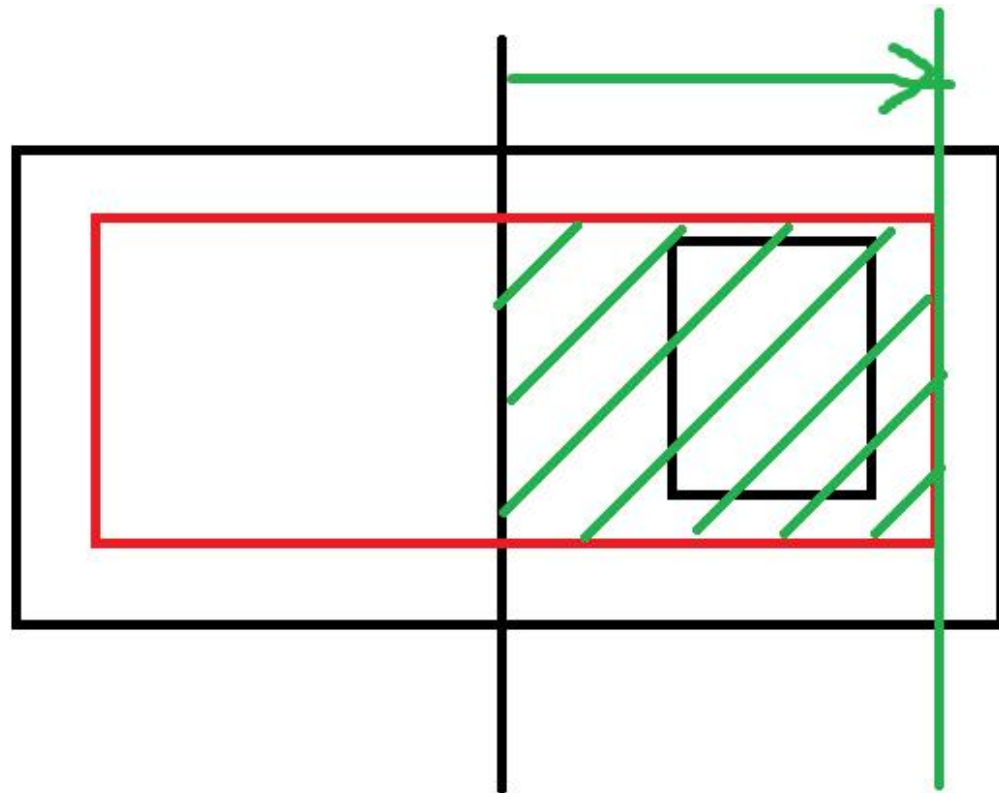
Solution

- 然后扫描线往两边分别扫，每次x维扫到新加入一个矩形的时候就进行一次y维上的区间加，每次x维扫出一个矩形的时候就进行一次y维上的区间减



Solution

- 我们需要维护的信息是y维的一个区间，在x维的一个前缀，上的信息合并，这道题中维护一个历史最大值就可以了，因为分治导致x维选的是一个前缀
- 每个矩形被拆成 $2 = O(1)$ 个查询
- 所以查询复杂度是 $O(q \log n)$ 的

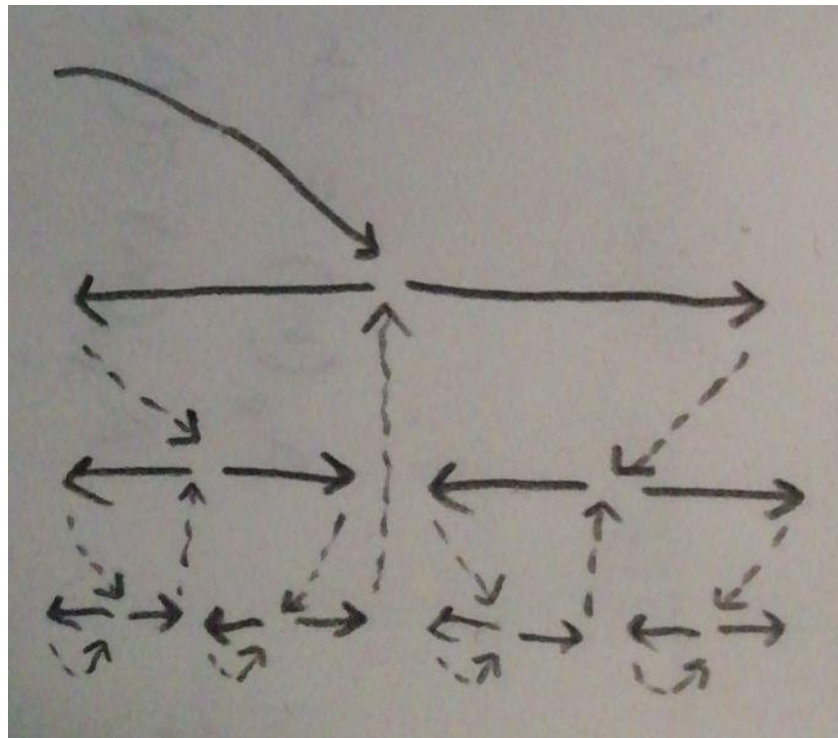


Solution

- 关于历史最大值的维护
- 区间加，维护区间历史最大值，维护二元组信息 (a, ha) 表示区间被加了 a ，历史最大值是 ha
- $(a, ha) * (b, hb) = (a + b, \max(ha, hb + a))$

Solution

- 考虑如何让分治的复杂度正确
- 可以和莫队类似，试着利用之前的信息
- 图中是分治的过程，其中虚线表示撤回一些修改



Solution

- 当撤回到虚线指的地方的时候，需要高效撤销之前算出来的区间历史最大值，因为我们虚线指的地方是一个分治中线
- 也就是说我们每次撤回到一个扫描线开始扫的位置，此时每个位置的最大值是跨过这个分治中线的矩形的最大值的合并
- 即我们要清空历史最大值，让其变成目前的最大值，这个可以通过在线段树上打一个特殊的标记来维护，即tag表示区间是否把历史最大值变成当前的最大值

Solution

- 可以证明刚才这个构造中，总的扫过的距离是 $O(n \log n)$ 的，而且每个矩形修改只会对应 $O(\log n)$ 次线段树上的区间修改
- 打特殊的标记总共 $O(n)$ 次
- 每个矩形询问被拆成了 $O(1)$ 次线段树上的区间历史最大值
- 总时间复杂度 $O((n+m) \log^2 n + q \log n)$
- 如果 n, m, q 同阶，可以将分治改成 $O(\log n / \log \log n)$ 叉的分治结构，这样的多叉平衡可以将复杂度平衡为 $O(n \log^2 n / \log \log n)$

Codeforces#493 E

- 给一个排列，查询区间中有多少子区间值域连续
- 注意读题

Solution

- 值域连续 $\Leftrightarrow [\max - \min = r - l]$
- 只需要维护区间内 $[\max - \min - (r - l) = 0]$ 的 (l, r) 个数即可
- 由于是排列, 所以 $(\max - \min) - (r - l) \geq 0$
- 于是可以将 0 个数的维护转换为 min 以及 min 个数的维护

Solution

- 单调栈搞一波之后，转换为：
- 平面上每个位置都有一个初始值为0的点，现在进行 $O(n)$ 次矩形加，操作完之后进行 m 次矩形min及min个数的查询
- 可以扫描线维护，转换为一维问题：区间历史min及min个数的维护
- 由于 $\text{min} \geq 0$
- 所以这个可以高效维护，多维护一下上次更新的时间即可

Thanks for listening